

Reviewer Pack — Minimal Rank of Quaternionic Forms Bounds Communication Comp...

Entry 2a696b6e9719 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-28 07:57:09 UTC

Minimal Rank of Quaternionic Forms Bounds Communication Complexity for XOR

Entry ID: 2a696b6e9719 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-28 07:57:09 UTC

1. Conjecture

Field A (mathematical branch): Quaternionic Geometry (Forms) **Field B** (complexity object): Communication Complexity

Statement:

[‘For every Boolean matrix $M \in \{\pm 1\}^{\{N \times N\}}$, the communication complexity $CC(XOR, M)$ is lower-bounded by $O(1 + \log_2(N \cdot \text{rank}_Q(M)))$, where $\text{rank}_Q(M)$ is the minimal rank of M when considered over the quaternionic field.’, ‘For all instances with $N \leq 40$, there exists an absolute constant $C > 0$ such that $CC(XOR, M) \geq C \cdot \log_2(N \cdot \text{rank}_Q(M))$.’, ‘For any Boolean matrix $M \in \{\pm 1\}^{\{N \times N\}}$, if $\text{rank}_Q(M) < \log_2(N)/4$ for $N \leq 40$, then $CC(XOR, M) \leq 3$.’]

Rationale (proposer’s reasoning):

[‘Quaternionic forms provide a non-commutative algebraic structure that may introduce new complexity-theoretic properties not present in classical arithmetic.’, ‘The minimal rank of quaternionic forms could potentially capture structural information about Boolean matrices that affects their communication complexity.’, ‘Previous work has established connections between higher-rank structures and communication complexity, suggesting that quaternionic geometry might provide a novel perspective.’]

Taxonomy category: QuaternionicGeometry (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): bacf5074bca2186d

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

If the Spearman rank correlation coefficient is ≥ 0.7 with at least 25 of 30 seeds, or if for any seed there exists an absolute constant $C > 1$ such that $CC(XOR, M) \geq C \cdot \log_2(N \cdot \text{rank}_Q(M))$ for all $N \leq 40$ and for a matrix with $\text{rank}_Q(M) < \log_2(N)/4$, then the conjecture is supported. Otherwise, it is falsified.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	HITS	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	1.00	UNCERTAIN	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "Quaternionic Geometry" AND "communication complexity" AND XOR - "minimal rank quaternionic forms" AND communication complexity - communication complexity AND XOR AND minimal rank Boolean matrix

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def generate_random_matrix(N):
    return [[random.choice([-1, 1]) for _ in range(N)] for _ in range(N)]

def matrix_rank(M):
    N = len(M)
    A = [row[:] for row in M]
    rank = 0
    for i in range(N):
        if all(A[j][i] == 0 for j in range(rank, N)):
            continue
        rank += 1
        pivot_row = rank - 1
        for j in range(i + 1, N):
            if A[j][i] != 0:
                factor = Fraction(A[j][i], A[pivot_row][i])
                for k in range(N):
                    A[j][k] -= factor * A[pivot_row][k]
    return rank

def communication_complexity(M):
    N = len(M)
    count = 0
    for i in range(N):
        for j in range(i + 1, N):
            if M[i][j] != M[j][i]:
                count += 1
```

```

    return count

def run_trial(seed: int) -> dict:
    random.seed(seed)
    results = []
    for n in [5, 10, 15, 20, 30, 40]:
        for _ in range(5): # Ensure at least 30 instances per seed
            M = generate_random_matrix(n)
            rank_Q = matrix_rank(M)
            CC = communication_complexity(M)
            results.append((n, rank_Q, CC))
    if not results:
        return {
            "metric_name": "communication_complexity",
            "metric_value": None,
            "instances_tested": 0,
            "conjecture_holds": False,
            "counterexample": "No instances generated"
        }

    metric_values = [CC * math.log2(n * rank_Q) for n, rank_Q, CC in results]
    mean_metric_value = sum(metric_values) / len(metric_values)
    std_metric_value = (sum((x - mean_metric_value) ** 2 for x in metric_values) / len(metric_values)) ** 0.5

    conjecture_holds = any(CC >= 1 * math.log2(n * rank_Q) for n, rank_Q, CC in results if rank_Q > 0)

    return {
        "metric_name": "communication_complexity",
        "metric_value": mean_metric_value,
        "instances_tested": len(results),
        "conjecture_holds": conjecture_holds,
        "counterexample": "" if conjecture_holds else "CC < C * log2(N * rank_Q) for some N and M"
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3
    results = []

    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {{\"seed\": {seed}, **trial_result}}")
        results.append(trial_result)

    mean_metric_value = sum(r["metric_value"] for r in results if r["metric_value"] is not None) / len(results)
    std_metric_value = (sum((r["metric_value"] - mean_metric_value) ** 2 for r in results if r["metric_value"] is not None) / len(results)) ** 0.5
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std={std_metric_value} support_fraction={support_fraction}")
    elif any(not r["conjecture_holds"] and r["counterexample"] != "" for r in results):
        first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"] and r["counterexample"] != "")
        print(f"RESULT: FALSIFIED counterexample=\"{results[0]['counterexample']}\" first_failing_seed={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE reason=insufficient_evidence")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_018bd0b7.py", line 84, in <module>
    trial_result = run_trial(seed)
                   ~~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_018bd0b7.py", line 52, in run_trial
    rank_Q = matrix_rank(M)
             ~~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_018bd0b7.py", line 32, in matrix_rank
    factor = Fraction(A[j][i], A[pivot_row][i])
             ~~~~~~
  File "/usr/lib/python3.12/fractions.py", line 281, in __new__
    raise ZeroDivisionError('Fraction(%s, 0)' % numerator)
ZeroDivisionError: Fraction(-2, 0)
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed due to a division by zero error, which prevented it from producing data necessary to evaluate the conjecture. | next: Investigate and fix the division by zero error in the code. Once fixed, rerun the test with appropriate seeds to verify the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	12017
2	preregistration	ollama_remote	glm4:latest	0	0	6992
3	novelty	ollama_remote	glm4:latest	0	0	4677
4	novelty	ollama_remote	glm4:latest	0	0	5550
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	38314
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11236
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10203
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11509
9	judge	ollama_remote	glm4:latest	0	0	12579

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 113076 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/2a696b6e9719.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/2a696b6e9719.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/2a696b6e9719.tar.gz` (if generated)